

Guide : Composition et Collections en Java

Encapsulation, Structures Imbriquées et Modélisation Objet-Collection

Introduction : Composition vs Héritage

En conception orientée objet, la **composition** (principe « a un » / *has-a*) consiste à inclure des objets (ou des collections d'objets) à l'intérieur d'une classe pour modéliser des relations structurelles complexes, plutôt que d'utiliser l'héritage (*is-a*).

Pourquoi privilégier la composition ?

- **Encapsulation stricte** : Masquer les structures de données internes et protéger l'intégrité des données métiers.
- **Couplage faible** : Permet de changer l'implémentation sous-jacente (ex : passer d'une `ArrayList` à un `HashSet`) sans casser le code client.
- **Flexibilité** : Modélisation fidèle des cardinalités du monde réel (1-à-1, 1-à-plusieurs, plusieurs-à-plusieurs).

Modélisation de Collections Imbriquées

Pour représenter des structures hiérarchiques ou multidimensionnelles, on combine souvent les interfaces de l'API Collections (ex : `Map` de `List`).

Exemple : Indexation par Catégorie

```
import java.util.*;

public class Catalogue {
    // Association : 1 Catégorie -> Plusieurs Produits
    private Map<String, List<String>> indexProduits = new HashMap<>();

    public void ajouterProduit(String categorie, String produit) {
        // computeIfAbsent initialise la liste si la cle n'existe pas
        indexProduits.computeIfAbsent(categorie, k -> new ArrayList<>()).add(
            produit);
    }

    public List<String> getProduits(String categorie) {
        return indexProduits.getOrDefault(categorie, Collections.emptyList());
    }
}
```

Encapsulation et Bonnes Pratiques

Exposer directement une collection mutable brise l'encapsulation. Il faut retourner des vues non modifiables ou des copies.

```
public class Universite {
    private String nom;
    private List<Etudiant> etudiants = new ArrayList<>();

    // Bonne pratique : Retourner une vue en lecture seule
    public List<Etudiant> getEtudiants() {
        return Collections.unmodifiableList(etudiants);
    }

    public void inscrire(Etudiant e) {
        if (e != null) { etudiants.add(e); }
    }
}
```

Exemple Global : Gestion d'une Équipe

```
import java.util.*;

public class EquipeProjet {
    String nomProjet;
    Set<String> membres = new HashSet<>();

    public EquipeProjet(String nomProjet) { this.nomProjet = nomProjet; }

    public boolean ajouterMembre(String membre) {
        return membres.add(membre); // Set gere l'unicite automatiquement
    }

    public void afficherMembres() {
        System.out.println("Projet : " + nomProjet + " | Membres : " + membres
            );
    }
}
```

Exercices Pratiques

Exercice 1 : Panier d'Achat (Niveau Facile)

1. Créez une classe `Article` (nom, prix).
2. Créez une classe `Panier` qui compose une collection d'articles (ex : `List<Article>`).
3. Ajoutez une méthode `double calculerTotal()` qui retourne la somme des prix des articles contenus dans le panier.

Exercice 2 : Annuaire Téléphonique Multi-numéros (Niveau Moyen)

1. Concevez une classe `Annuaire` qui utilise en interne une `Map<String, List<String>` associant un nom de contact à une liste de numéros de téléphone.
2. Implémentez une méthode `ajouterNumero(String nom, String numero)` qui gère l'ajout d'un numéro pour un contact existant ou nouveau.
3. Implémentez une méthode pour lister tous les numéros d'un contact donné.

Exercice 3 : Gestionnaire de Cours et Étudiants (Niveau Avancé)

1. Créez une classe `Ecole` qui maintient une correspondance entre des identifiants de cours (`String`) et les étudiants inscrits (`Set<Etudiant>`).
2. Garantisiez qu'un étudiant ne peut pas être inscrit deux fois au même cours (grâce aux propriétés du `Set` et à une redéfinition correcte de `equals/hashCode` dans `Etudiant`).
3. Proposez une méthode pour trouver les cours communs à un étudiant.